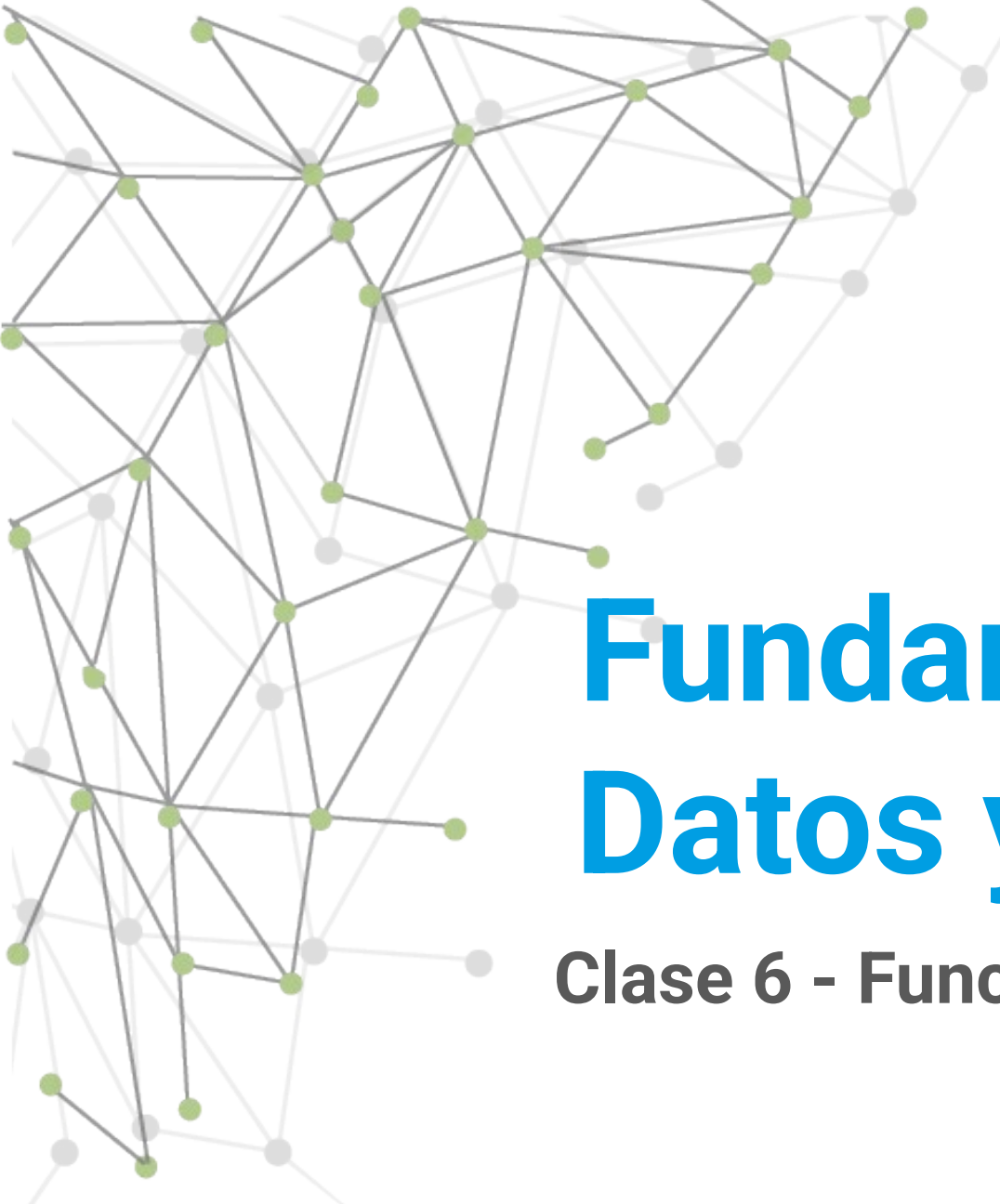
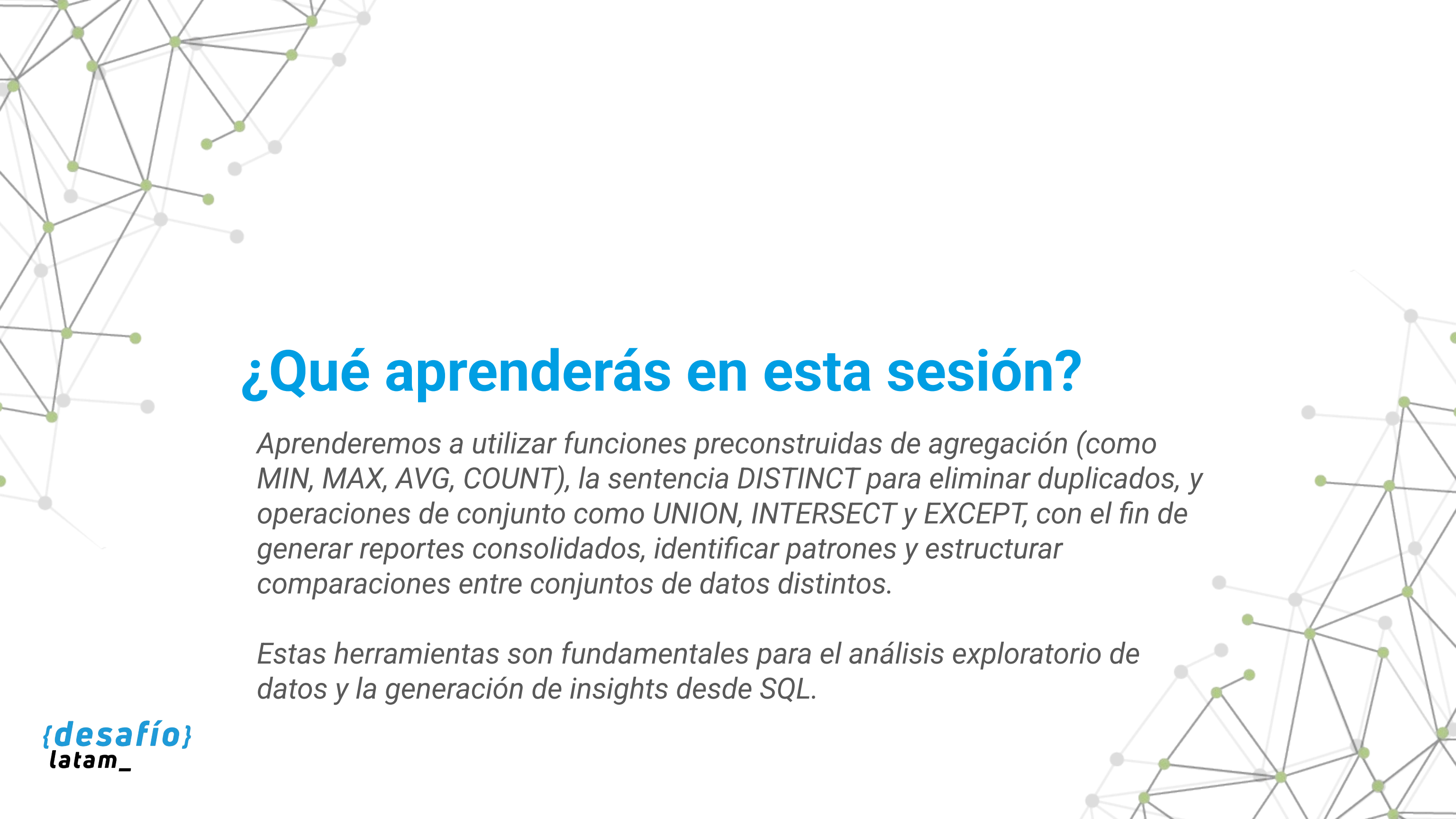




Fundamentos de Bases de Datos y Consultas Básicas

Clase 6 - Funciones y operaciones de conjunto en SQL



¿Qué aprenderás en esta sesión?

Aprenderemos a utilizar funciones preconstruidas de agregación (como MIN, MAX, AVG, COUNT), la sentencia DISTINCT para eliminar duplicados, y operaciones de conjunto como UNION, INTERSECT y EXCEPT, con el fin de generar reportes consolidados, identificar patrones y estructurar comparaciones entre conjuntos de datos distintos.

Estas herramientas son fundamentales para el análisis exploratorio de datos y la generación de insights desde SQL.

Desafío inicial

Trabajas como analista en una compañía de retail con presencia nacional. El gerente de ventas necesita un reporte consolidado con los siguientes requerimientos:

- Promedio, máximo y mínimo de ventas mensuales por región.
- Conteo total de transacciones registradas durante el último trimestre.
- Listado sin duplicados de productos vendidos.
- Comparación entre las tiendas del norte y del sur para identificar productos vendidos exclusivamente en una de las zonas.

Actualmente, la información está almacenada en varias tablas y los reportes generados manualmente suelen tener inconsistencias. Para cumplir con este requerimiento de forma rápida, precisa y automatizable, deberás dominar funciones agregadas, sentencias para eliminar duplicados y operaciones entre conjuntos.

Durante esta sesión desarrollarás consultas SQL que te permitirán resolver cada uno de estos puntos con eficiencia y aplicación directa en entornos reales.

Funciones preconstruidas (MIN, MAX, AVG, COUNT)

¿Cómo puedes obtener estadísticas rápidas de tus datos sin tener que programar cálculos complejos?

Las funciones preconstruidas o funciones agregadas en SQL permiten obtener resúmenes estadísticos directamente desde las consultas, sin necesidad de realizar operaciones manuales o utilizar herramientas externas. Son ampliamente utilizadas para generar reportes ejecutivos, tableros de control y análisis de comportamiento en bases de datos de diversa índole (comerciales, académicas, industriales, entre otras).

Función MIN() y MAX()

Estas funciones devuelven el valor mínimo o máximo de una columna numérica, alfabética o de fecha.

Ejemplo aplicado

```
SELECT  
    MIN(fecha_compra) AS primera_compra,  
    MAX(fecha_compra) AS ultima_compra  
FROM ventas;
```

Este reporte es útil para analizar la vigencia de clientes o determinar los periodos activos de compra.

Contexto productivo

Aplicaciones

Determinar el rango de actividad de los clientes en e-commerce, identificar el tiempo de respuesta más bajo o más alto en centros de soporte técnico.

Buenas prácticas

- Usar alias (AS) para claridad en los encabezados de salida.
- Aplicar sobre columnas indexadas para mejorar el rendimiento.

Errores comunes

Aplicar MIN() o MAX() sobre campos de tipo texto sin validar su formato puede llevar a resultados inesperados.

Función AVG()

Calcula el promedio de una columna numérica. Es clave en análisis de desempeño, precios, productividad, etc.

Ejemplo aplicado

```
SELECT
    AVG(monto_total) AS promedio_ventas
FROM ventas
WHERE estado = 'Finalizada';
```

Contexto productivo

Aplicaciones

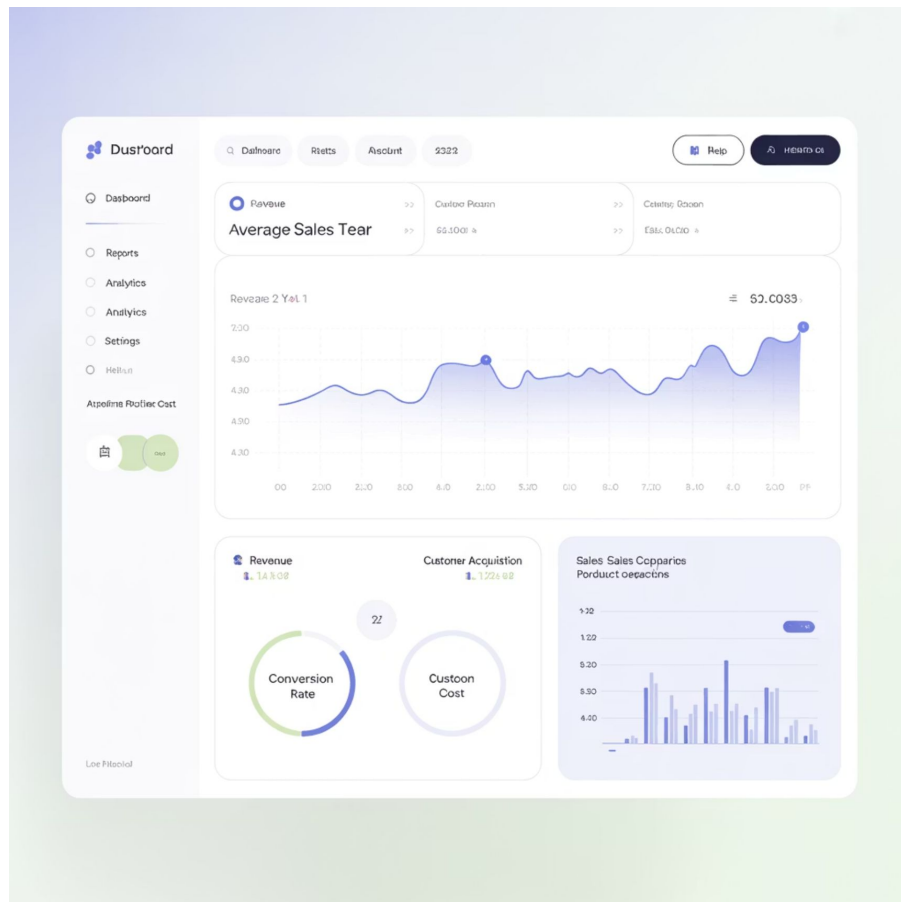
En un entorno comercial, permite evaluar la tendencia de compra y establecer objetivos por campaña o temporada.

Buenas prácticas

- Filtrar previamente con WHERE para evitar distorsión con valores atípicos o registros incompletos.
- Usar CASE WHEN para segmentar promedios por condiciones específicas.

Errores comunes

No considerar NULL, lo que puede alterar el resultado promedio si no se filtran adecuadamente.



Función COUNT()

Devuelve el número de registros o valores distintos, según cómo se use.

Ejemplos aplicados

```
-- Total de ventas registradas
```

```
SELECT COUNT(*)
```

```
FROM ventas;
```

```
-- Total de ciudades distintas
```

```
SELECT COUNT(DISTINCT ciudad)
```

```
FROM clientes;
```

Buenas prácticas y errores comunes

Errores comunes

Contar con COUNT(campo) sin considerar los NULL, lo que puede excluir registros esperados.

Buenas prácticas

- Utilizar COUNT(*) para contar todas las filas.
- Aplicar COUNT(DISTINCT campo) para auditar valores únicos.

Métricas clave

- Total de registros válidos
- Porcentaje de duplicados
(combinando con COUNT(DISTINCT) y COUNT(*))
- Distribución por segmentos usando GROUP BY

Combinación con GROUP BY

Permite generar estadísticas agrupadas por categorías o segmentos de negocio.

Ejemplo aplicado

```
SELECT
    region,
    COUNT(*) AS total_clientes,
    AVG(ingreso_mensual) AS ingreso_promedio,
    MAX(ingreso_mensual) AS ingreso_maximo
FROM clientes
GROUP BY region;
```

Contexto productivo

Análisis comparativo por zonas geográficas, sucursales, líneas de productos.

Errores comunes y buenas prácticas

Errores comunes

- Usar columnas no incluidas en GROUP BY sin funciones agregadas.
- Agrupar sin revisar consistencia en los datos fuente (ej. regiones mal escritas).

Buenas prácticas

- Revisar ortografía y tipificación previa de los datos.
- Utilizar HAVING para aplicar condiciones sobre los valores agregados.

Métrica clave

Desviación entre máximos y promedios por grupo, útil para análisis de dispersión.

Como hemos visto, las funciones preconstruidas permiten analizar rápidamente tendencias, comportamientos y métricas clave. Sin embargo, muchas veces los datos pueden contener registros duplicados que distorsionan estas métricas. Por eso, el siguiente paso será conocer cómo identificar y excluir duplicados mediante la sentencia DISTINCT, y cómo esto impacta en la calidad y precisión de nuestros análisis.

La sentencia DISTINCT

¿Cómo asegurar que los datos que analizas realmente representan valores únicos y no están inflados por repeticiones?

La sentencia DISTINCT en SQL permite eliminar duplicados de los resultados de una consulta, mostrando únicamente valores únicos para una o varias columnas. Es especialmente útil cuando se trabaja con bases de datos que registran múltiples eventos por cliente, producto o variable, y se requiere un conteo o listado sin repeticiones.



Uso básico de DISTINCT

Ejemplo aplicado

```
SELECT DISTINCT ciudad  
FROM clientes;
```

Este ejemplo devuelve todas las ciudades distintas registradas en la tabla de clientes, lo cual puede ser útil para construir un filtro de regiones o estimar cobertura geográfica.

Contexto productivo

- Identificar las zonas de operación activa.
- Generar listados únicos de proveedores, productos, o fechas clave para análisis operativo.

Buenas prácticas y errores comunes

Buenas prácticas

- Asegurarse de que el campo consultado no contenga errores de digitación que puedan parecer únicos pero no lo son (ej. "Santiago" vs. "santiago").
- Usar ORDER BY para mejorar la legibilidad en listados grandes.

Errores comunes

- Aplicar DISTINCT sobre muchas columnas puede generar resultados inesperados si hay una ligera variación en alguna de ellas.
- Asumir que DISTINCT limpia datos mal ingresados. No corrige errores, solo evita mostrar duplicados exactos.

DISTINCT con múltiples columnas

Ejemplo aplicado

```
SELECT DISTINCT  
    ciudad,  
    estado_cuenta  
FROM clientes;
```

Este ejemplo devuelve combinaciones únicas de ciudad y estado de cuenta, útil para segmentar cartera o evaluar la distribución de condiciones comerciales.

Métrica clave

Número de combinaciones únicas relevantes para segmentaciones (ej. región-estado, producto-cliente).

Comparación con GROUP BY

Aunque ambas opciones pueden usarse para obtener registros únicos, DISTINCT devuelve solo los datos sin agregación, mientras que GROUP BY permite aplicar funciones agregadas.

Ejemplo comparativo

```
-- Usando DISTINCT
SELECT DISTINCT ciudad
FROM clientes;

-- Usando GROUP BY
SELECT ciudad
FROM clientes
GROUP BY ciudad;
```

Contexto productivo

- DISTINCT para limpiar vistas o exportaciones.
- GROUP BY cuando se desea contar, promediar o sumar por grupo.

Errores comunes y buenas prácticas

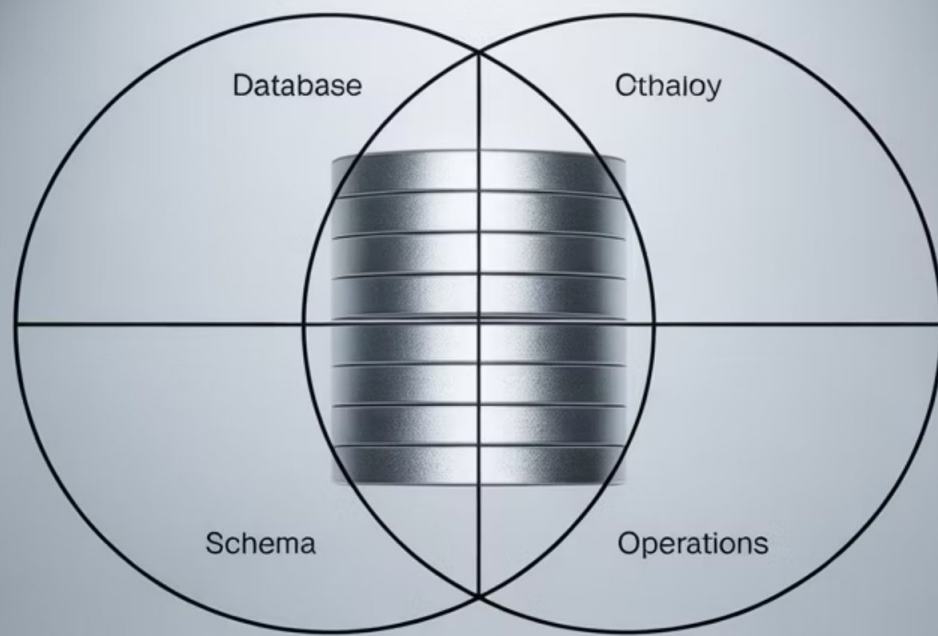
Errores comunes

Usar DISTINCT cuando realmente se necesita una agregación.

La sentencia DISTINCT es una herramienta clave para mejorar la precisión en consultas exploratorias o listados sin repeticiones. Sin embargo, cuando es necesario combinar resultados de distintas consultas, filtrar intersecciones o excluir datos, necesitamos ir más allá. Por ello, en el siguiente tema aprenderás a usar UNION, INTERSECT y EXCEPT, operaciones que permiten combinar o contrastar conjuntos de datos de forma más avanzada.

Buenas prácticas

Usar GROUP BY cuando se desee hacer cálculos, y DISTINCT solo para limpieza visual.



Operaciones UNION, INTERSECT y EXCEPT

¿Cómo puedes comparar o combinar resultados de múltiples consultas en SQL para obtener una vista consolidada o diferenciada de los datos?

En análisis complejos o en integración de datos desde múltiples fuentes, las operaciones de conjunto en SQL (UNION, INTERSECT, EXCEPT) permiten combinar resultados de varias consultas SELECT, identificando coincidencias, diferencias o sumas. Son fundamentales para consolidar información, eliminar duplicados o contrastar escenarios.

Uso de UNION

UNION permite combinar los resultados de dos o más consultas y elimina duplicados por defecto.

Ejemplo aplicado

```
SELECT
    nombre,
    correo
FROM clientes_nuevos
UNION
SELECT
    nombre,
    correo
FROM clientes_antiguos;
```

Este ejemplo genera un listado consolidado de clientes, evitando repetir registros que aparecen en ambas tablas.

Contexto productivo y buenas prácticas

Contexto productivo

- Consolidación de listas de contactos de distintas campañas.
- Unión de registros de distintas fuentes temporales (ej. reportes semanales).

Buenas prácticas

Asegurar que las columnas tengan los mismos tipos y el mismo número de columnas en cada SELECT.

Errores comunes

- Diferencias en tipo de dato o número de columnas entre consultas.
- Suponer que se conservarán duplicados; para eso se usa UNION ALL.

Uso de INTERSECT

INTERSECT devuelve los registros que están presentes en ambas consultas.

Ejemplo aplicado

```
SELECT nombre  
FROM clientes_2023  
INTERSECT  
SELECT nombre  
FROM clientes_2024;
```

Este ejemplo muestra los clientes que han estado activos en ambos años, útil para programas de fidelización.

Contexto productivo y buenas prácticas

Contexto productivo

- Identificar clientes que han permanecido en dos periodos.
- Determinar elementos comunes entre campañas o listas de distribución.

Errores comunes

No considerar mayúsculas/minúsculas si la base de datos distingue entre ellas.

Buenas prácticas

Aplicar LOWER() o UPPER() si se requiere una intersección insensible a mayúsculas.

Uso de EXCEPT

EXCEPT muestra los registros de la primera consulta que no aparecen en la segunda.

Ejemplo aplicado

```
SELECT correo
FROM clientes_suscritos
EXCEPT
SELECT correo
FROM clientes_inactivos;
```

Este ejemplo permite identificar contactos válidos para próximas campañas, eliminando a quienes ya no están activos.

Contexto productivo y buenas prácticas

Contexto productivo

Filtrado de listas para evitar
reprocesamiento o envíos erróneos.

Buenas prácticas

Verificar que los criterios de exclusión
estén bien definidos y validados.

Errores comunes

Esperar que EXCEPT funcione sobre
columnas de tipo complejo (JSON, BLOB,
etc.).

Las operaciones UNION, INTERSECT y EXCEPT amplían nuestra capacidad para analizar y gestionar datos provenientes de múltiples orígenes o con múltiples condiciones. Dominar estas herramientas nos permite no solo comparar conjuntos, sino también optimizar la construcción de reportes y segmentaciones avanzadas en contextos empresariales reales.

Actividad guiada:

**Consolidando reportes con
funciones y operaciones de
conjunto**



Objetivo de la actividad

Aplicar funciones preconstruidas y operaciones de conjunto (UNION, INTERSECT, EXCEPT) para construir reportes y realizar análisis comparativos entre conjuntos de datos, simulando un escenario real de consolidación de información comercial.



Situación problema

Eres parte del equipo de análisis de datos de una empresa de retail que administra campañas para clientes nuevos y antiguos.

Debes generar un informe consolidado con datos clave: total de clientes únicos, ingresos promedio, ciudades con cobertura y segmentaciones entre clientes activos y no activos.

Además, deberás identificar coincidencias y diferencias entre bases de datos de distintos períodos de campaña.



Script base

```
-- Tabla: clientes_2023
CREATE TABLE clientes_2023 (
  nombre VARCHAR(100),
  correo VARCHAR(100),
  ciudad VARCHAR(50),
  ingreso_mensual DECIMAL(10,2)
);

INSERT INTO clientes_2023 VALUES
  ('Ana', 'ana@mail.com', 'Santiago', 1100000),
  ('Luis', 'luis@mail.com', 'Valparaíso', 950000),
  ('Carlos', 'carlos@mail.com', 'Rancagua', 1050000);
```

```
-- Tabla: clientes_2024
CREATE TABLE clientes_2024 (
  nombre VARCHAR(100),
  correo VARCHAR(100),
  ciudad VARCHAR(50),
  ingreso_mensual DECIMAL(10,2)
);

INSERT INTO clientes_2024 VALUES
  ('Ana', 'ana@mail.com', 'Santiago', 1150000),
  ('Camila', 'camila@mail.com', 'Concepción', 980000),
  ('Carlos', 'carlos@mail.com', 'Rancagua', 1100000);
```

Consultas guiadas

1

Obtener ciudades distintas de ambas campañas

```
SELECT DISTINCT ciudad
FROM clientes_2023
UNION
SELECT DISTINCT ciudad
FROM clientes_2024;
```

2

Calcular ingreso promedio 2024

```
SELECT
    AVG(ingreso_mensual) AS ingreso_promedio
FROM clientes_2024;
```

3

Identificar clientes en ambas campañas

```
SELECT nombre
FROM clientes_2023
INTERSECT
SELECT nombre
FROM clientes_2024;
```

4

Clientes 2023 que no están en 2024

```
SELECT nombre
FROM clientes_2023
EXCEPT
SELECT nombre
FROM clientes_2024;
```

Instrucciones

1. Ejecuta el script base en tu entorno SQL (puede ser SQLite, PostgreSQL o entorno simulado).
2. Analiza los resultados de las consultas paso a paso.
3. Modifica las tablas para agregar un nuevo cliente en 2024 y observa el impacto en UNION e INTERSECT.
4. Justifica en tu cuaderno digital por qué utilizarías DISTINCT, y en qué caso preferirías GROUP BY.
5. Discute en grupo cómo combinar estos resultados en reportes dinámicos o dashboards ejecutivos.

Materiales necesarios

- Script SQL base entregado por el relator.
- Entorno SQL local o en la nube (SQLite, DBeaver, PostgreSQL, MySQL, etc.).
- Hoja de trabajo o cuaderno digital para análisis y justificación.



Resumen

-  Comprendimos qué son las funciones preconstruidas en SQL (MIN, MAX, AVG, COUNT) y cómo se aplican a consultas de resumen.
-  Aplicamos la sentencia DISTINCT para eliminar duplicados y entender su efecto en conjuntos de datos.
-  Exploramos el uso de operaciones de conjunto (UNION, INTERSECT, EXCEPT) para comparar resultados entre tablas relacionadas o versiones históricas.
-  Analizamos aplicaciones productivas en entornos de inteligencia comercial, campañas de marketing y consolidación de reportes.
-  Evaluamos buenas prácticas para garantizar precisión en los resultados y minimizar errores por estructura o formato de los datos.



Próxima sesión...

En la siguiente sesión aprenderemos a realizar consultas sobre tablas relacionadas, interpretando modelos de datos y aplicando principios de integridad referencial.

Conoceremos cómo funcionan las llaves primarias y foráneas.

Preguntas de cierre

1

Pregunta 1

¿Qué diferencias hay entre COUNT(*) y COUNT(columna)?

2

Pregunta 2

¿Cuál es el riesgo de no aplicar DISTINCT cuando se consolidan registros de distintas fuentes?

3

Pregunta 3

¿En qué casos usarías UNION y cuándo INTERSECT?

4

Pregunta 4

¿Qué errores pueden surgir al usar EXCEPT si las columnas no coinciden?

5

Pregunta 5

¿Cómo contribuyen estas funciones a la generación de reportes ejecutivos confiables?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2019). Database systems: Design, implementation, & management (13th ed.). Cengage Learning.
- Pratt, P. J., & Last, M. Z. (2014). Concepts of Database Management (8th ed.). Cengage Learning.
- IBM. (s.f.). SQL reference. IBM Knowledge Center. <https://www.ibm.com/docs/en/db2>
- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. <https://www.postgresql.org/docs/>